PROJECT CRYGAN

A Cryptographic Framework for Tamper-Evident Video Evidence Recording and Verification

Submitted by
S VINAY NARASIMHA
R HEMANTH KUMAR

Version 1.0
July 2026

Table of Contents

1. Executive Summary

Project Crygan is a desktop application that records video and, at the same time, builds an independently verifiable, cryptographically signed record of that recording's integrity. The goal is to make it possible to later answer two questions with strong technical evidence: has this specific video file been altered since it was recorded, and if so, exactly which parts of it changed.

The system combines several well-established cryptographic and forensic techniques into a single, layered pipeline: a rolling SHA-256 hash chain over every recorded frame, a Merkle tree for pinpointing altered frames, perceptual hashing to distinguish harmless re-encoding from genuine visual tampering, ECDSA digital signatures, AES-256-GCM encryption of evidence metadata, RFC 3161 trusted timestamping, and four independent, redundant storage mechanisms for the resulting evidence package.

This report documents the complete architecture of the project — every module, how they interact, and the reasoning behind each design decision — followed by a structured account of the functional testing performed against the system, covering unmodified recordings, trimmed video, visual (exposure/saturation) tampering, and tampering of a secondary evidence artifact. The results confirm that the system reliably distinguishes authentic recordings from altered ones, localizes altered frame indices and provides supporting classification of the alteration, and degrades gracefully when individual evidence artifacts are lost or damaged, without ever falsely validating a tampered recording.

2. Introduction

2.1 Background and Motivation

Video is routinely relied upon as evidence — in legal disputes, journalism, human rights documentation, and everyday incident reporting — yet standard video files carry no built-in guarantee that they represent an unaltered recording. Once a video leaves the device that captured it, there is normally no way for a third party to confirm that no frames were removed, no visual content was altered, and no metadata was falsified, short of trusting the person presenting the file.

Existing efforts to address this — such as the C2PA (Coalition for Content Provenance and Authenticity) standard adopted by some camera manufacturers, and human-rights-focused tools such as ProofMode and eyeWitness to Atrocities — demonstrate that cryptographically-backed provenance is both feasible and valuable, but each makes different design trade-offs around server dependency, granularity of tamper detection, and resilience to re-encoding.

Project Crygan was built to explore this problem space directly: to design and implement, end-to-end, a tamper-evidence pipeline that works entirely offline, gives frame-level (rather than only file-level) tamper feedback, and stores its evidence redundantly enough to survive common real-world damage such as re-encoding, partial file loss, or accidental deletion of a companion file.

2.2 Objectives

1. Design and implement a cryptographically sound pipeline that signs and encrypts a video's integrity metadata at the moment of recording.
2. Detect, with high confidence, whether a recorded video has been altered after the fact.
3. When alteration is detected, localize it — identify which frames changed and classify the nature of the change (ordinary re-encoding versus a genuine visual modification).
4. Ensure the evidence needed to perform this verification survives common forms of damage: re-encoding, transcoding, partial file corruption, or loss of any single storage artifact.
5. Operate entirely on the local machine, without any dependency on a remote server or third-party service for its core functionality.
6. Produce human-readable verification reports suitable for review by a non-technical stakeholder.

2.3 Scope

This project covers the full pipeline from video capture through evidence generation, storage, recovery, and verification, implemented as a cross-platform desktop application. It does not attempt to address the separate and substantially harder problem of proving that the camera sensor itself was not fed fabricated footage (e.g., via a virtual camera device) — this is a known, open limitation shared by every tool in this space, discussed further in Section 5.3.

3. System Architecture Overview

3.1 Project Structure

The application is organized into five Python modules, each with a distinct and non-overlapping responsibility:

File	Responsibility
main.py	Application entry point. Initializes the Qt application, applies the saved theme, and launches the main window. Contains no business logic.
config.py	Central configuration: all filesystem paths, cryptographic parameters, thresholds, and feature flags used by every other module.
crypto_core.py	All cryptography: key generation/management, the frame hash chain, Merkle tree construction, perceptual hashing, digital signatures, encryption/decryption, RFC 3161 timestamping, and the multi-stage VerificationEngine.
evidence_storage.py	All evidence packaging and recovery: tail-appended embedding inside the video file, the companion .crygan package, the LSB steganographic reference frame, Reed–Solomon erasure-coded chunk frames, and the evidence-export bundler.
project_UI.py	The GUI and the application/session logic that supports it: the SQLite evidence database, the camera recorder, IP-based location lookup, PDF report generation, and the four main screens (Record, Verify, Reports, Settings).

Table 3.1 — Module responsibilities.

3.2 High-Level Data Flow

The system operates in two distinct phases, described below.

Recording Phase

7. The user starts a recording; frames are captured from the selected camera device and written to an MP4 file.
8. As each frame is captured, its SHA-256 hash is computed and folded into a running (chained) hash, and separately recorded as a leaf of a Merkle tree.
9. On stopping the recording, the final chain hash and Merkle root are computed, an ECDSA signature is produced over them, and (optionally) an RFC 3161 trusted timestamp is requested for the final hash.
10. A structured evidence record — final hash, Merkle root, frame count, GPS coordinates, timestamp, signature, and camera metadata — is serialized to JSON and encrypted with AES-256-GCM.
11. This encrypted evidence package is then stored redundantly through four independent mechanisms, described in detail in Section 4.4: appended to the end of the video file itself, written to a companion .crygan file, split across Reed–Solomon erasure-coded PNG frames, and fingerprinted into a local registry. Every recording is also assigned its own unique evidence identifier (a UUID), which is embedded directly into the reference PNGs alongside the payload data; because recovery is keyed off this identifier rather than the video's filename, evidence discovery continues to work correctly even if the video is later renamed or moved (see Section 4.4.6).

Verification Phase

12. The user selects a video (and, optionally, a passphrase and any recovered reference artifacts) and runs verification.
13. The application attempts to locate and decrypt the evidence package, trying each storage mechanism in priority order until one succeeds: (1) embedded evidence within the video file, (2) the companion .crygan package, (3) Reed–Solomon chunk-frame recovery, and (4) the local out-of-band registry (see Section 4.4 for the mechanism behind each stage).
14. Every frame of the video under test is re-read, re-hashed, and re-chained; the recomputed final hash and Merkle root are compared against the recorded ones.
15. If the chain doesn't match, the Merkle tree is used to identify exactly which frame indices differ, and perceptual hashing classifies each differing frame as either an ordinary re-encoding artifact or a genuine visual change.
16. The digital signature is verified against the recorded public key, and (if present) the RFC 3161 timestamp and LSB reference frame are cross-checked as independent, corroborating signals.
17. A structured VerificationResult is produced, which drives both the on-screen results panel and the exportable PDF report.

4. Detailed Module Description

4.1 config.py — Central Configuration

This module centralizes every constant used elsewhere in the application, so that no other module hard-codes a path, threshold, or cryptographic parameter. Key configuration groups include:

- Filesystem layout: all persistent data (keys, videos, reports, steganographic reference frames, evidence exports, and the SQLite database) is stored in ordinary sub-folders next to the application itself, rather than in an OS-level app-data directory — keeping everything visible and easy to inspect or back up directly.
- Cryptographic parameters: ECC curve (SECP384R1), AES key size (256-bit), PBKDF2 iteration count (390,000), and salt/nonce sizes.
- Reed–Solomon parameters: 6 data chunks plus 4 parity chunks by default, meaning any 6 of the 10 resulting reference frames are sufficient to fully reconstruct the evidence package.
- Perceptual hashing thresholds: a 64-bit dHash per frame, with a 10-bit Hamming-distance threshold used to classify a mismatched frame as "plausibly re-encoded" rather than "visually altered."
- RFC 3161 timestamp authority configuration and IP-based geolocation endpoint.

4.2 main.py — Application Bootstrap

A minimal entry point: it configures logging, constructs the Qt application, applies the user's saved theme, and shows the main window. It intentionally contains no cryptographic or evidence-handling logic of its own.

4.3 crypto_core.py — Cryptography and Verification Engine

4.3.1 Key Management

Each installation generates its own ECDSA key pair on curve SECP384R1. The private key is never stored in plaintext: it is encrypted with a key derived from the user's session password via PBKDF2-HMAC (390,000 iterations) and stored as an AES-256-GCM-encrypted blob, alongside its salt. The public key is stored separately in plaintext, since it must be distributable for verification.

4.3.2 Frame Hash Chain

During recording, every frame's raw decoded pixel data is hashed with SHA-256, and each hash is chained to the previous one (hash of [previous_chain_value + current_frame_hash]), producing a single final value that depends on every frame in both content and order. During verification, the same process is repeated on the video under test; any change to any frame, or to the frame sequence itself, produces a different final chain value.

Hashing decoded pixel data rather than raw file bytes is a deliberate design choice: it avoids dependence on the container format or compressed bitstream, so a file that is merely re-muxed or re-packaged without touching pixel values still verifies cleanly. However, as observed in Test Case 2 (Section 6.3), ordinary re-encoding through a different codec typically does alter decoded pixel values enough to change the resulting SHA-256 frame hashes, so the exact-hash chain by itself still fails after re-encoding. The perceptual-hash layer described below is what actually lets the system distinguish an

ordinary transcoding artifact from a genuine content change once the exact-hash chain has already flagged a discrepancy.

4.3.3 Merkle Tree Tamper Localization

In addition to the linear chain, each frame's own (unchained) hash is used as a leaf in a Merkle tree. This provides two capabilities the linear chain alone cannot: first, if the chain check fails, the Merkle tree lets the verifier report exactly which frame indices differ, rather than only "integrity check failed" with no further detail; second, it allows a single frame to be proven part of the original recorded sequence using only that frame and a short Merkle proof, without needing the entire video file.

4.3.4 Perceptual Hashing (dHash)

For every frame flagged as differing by the exact-hash comparison, a 64-bit perceptual hash (dHash) is computed for both the recorded and recomputed frame, using only OpenCV and NumPy. Because perceptual hashes change gradually with visual similarity rather than catastrophically with any single bit change (unlike SHA-256), the Hamming distance between the two hashes lets the system classify the discrepancy: a small distance (≤10 of 64 bits, by default) indicates an ordinary re-encoding artifact, while a large distance indicates the visual content actually changed.

4.3.5 RFC 3161 Trusted Timestamping

The recording's own device clock can be altered by anyone before recording, so its "timestamp" claim is inherently weak. To strengthen this, the final hash chain value is optionally submitted to a public RFC 3161 Time-Stamp Authority, which returns a token independently and cryptographically attesting that this exact hash existed by a specific time, according to the authority's own clock — not the recording device's.

4.3.6 VerificationEngine

This is the central orchestrator of the verification phase. It attempts to locate a valid evidence package through each storage mechanism in priority order (embedded copy, companion file, Reed–Solomon chunk reconstruction, local registry fallback), decrypts and validates it, recomputes the frame hash chain and Merkle root from the video under test, performs the perceptual-hash classification of any mismatched frames, verifies the digital signature, and cross-checks the optional LSB reference frame and RFC 3161 timestamp. All findings are assembled into a single structured VerificationResult object.

4.4 evidence_storage.py — Evidence Packaging and Recovery

This module is responsible for making the encrypted evidence package survive as many forms of real-world damage as reasonably possible, through four independent and redundant mechanisms, plus a fifth out-of-band fallback.

4.4.1 Tail-Appended Embedding

The primary mechanism: the encrypted evidence package is appended after the video file's own end-of-file marker, wrapped in a way that does not disturb normal playback, and additionally mirrored into a standard ISO-BMFF 'uuid' box inside the MP4 container structure itself. This is the fastest path to verify, but does not survive re-encoding/transcoding, since that process rewrites the container and compressed bitstream from scratch.

4.4.2 Companion .crygan Package

A complete, independent copy of the encrypted evidence package is also written to a sidecar file (same name as the video, .crygan extension) in the same folder. This survives any editing done to the video file itself, as long as the sidecar file is kept alongside it.

4.4.3 LSB Steganographic Reference Frame

A single frame from the recording is used to hide, via classical least-significant-bit steganography, a small reference record: just the final hash and the digital signature (not the full evidence package). This is an independent corroborating cross-check — if it validates, it strongly reinforces confidence that the file wasn't tampered with — but it is not, by design, a way to reconstruct lost evidence. It is intentionally not designed for evidence recovery; its sole purpose is to provide an independent integrity corroboration alongside the primary evidence package, distinct from the recovery role played by the companion .crygan file and the Reed–Solomon chunk frames described next.

4.4.4 Reed–Solomon Erasure-Coded Chunk Frames

The full encrypted evidence package is split into 6 equal data chunks, plus 4 Reed–Solomon parity chunks (10 total by default), each hidden via LSB steganography into its own lossless PNG frame. Because of how Reed–Solomon erasure coding works, any 6 of these 10 frames — in any combination — are sufficient to fully reconstruct the original package, even if up to 4 are lost, deleted, or corrupted.

4.4.5 Local Perceptual-Hash Registry

As a last-resort, out-of-band fallback, a compact perceptual fingerprint of the recording (evenly, relative-position-sampled dHash values) is stored alongside a redundant copy of the encrypted evidence package in the application's local SQLite database. If a submitted video has no extractable embedded evidence at all — for instance, after a full transcode that also destroyed the companion file and all reference PNGs — verification can fall back to matching the video's fingerprint against this registry. This is explicitly weaker provenance than any of the in-file mechanisms above, since it depends on trusting this specific installation's local database rather than purely on the cryptographic chain, and is always reported as such.

4.4.6 Evidence-ID-Based Discovery

Each recording is assigned its own unique evidence identifier (a UUID) at the moment it is created. This identifier is embedded directly into the hidden pixel data of both the LSB reference frame and every Reed–Solomon chunk frame. During verification, all reference PNGs in the storage folder are scanned and grouped by this embedded identifier, rather than by matching the video's current filename. This design specifically ensures that recovery continues to work correctly even if the video file is later cropped, trimmed, renamed, or moved by an external tool — a scenario under which pure filename-based matching would otherwise fail to locate reference material that is, in fact, still fully present and valid.

4.4.7 Evidence Export Bundling

A one-step export function gathers every artifact discovered for a given recording — the video, its companion file, its LSB reference frame, and its Reed–Solomon chunk frames — together with a manifest describing exactly what was included, into a single, self-contained folder named after that

recording's evidence identifier. This is intended to make sharing or archiving a complete evidence set straightforward, rather than requiring several loose files to be located and transferred individually.

4.4.8 Evidence Package Contents

The reviews above repeatedly refer to the “encrypted evidence package” without listing exactly what it contains. Before any of the four storage mechanisms are applied, the recorder assembles a single structured record and serializes it to canonical JSON. That record holds:

- Application version (app_version), so an older evidence format can be recognized during long-term verification.
- Location block: latitude, longitude, resolution source (precise Windows Location Services or approximate IP geolocation), resolved city/country, accuracy in meters where available, and the UTC time the location was resolved (see Section 4.5.4).
- Timestamp block: local recording date, local time, UTC offset, and the full UTC start time in ISO 8601 form.
- Frame hash chain block: the hashing algorithm identifier, total frame count, the final chained SHA-256 hash, the Merkle algorithm identifier, the Merkle root, and the base64-encoded per-frame leaf hashes.
- Perceptual hash block (when the optional perceptual-hashing dependency is available): the dHash algorithm identifier and the base64-encoded per-frame perceptual hashes.
- Camera metadata: information about the recording device (e.g. resolution, frame rate, and, where available, the camera's reported name).
- RFC 3161 trusted timestamp block, when a Time-Stamp Authority was reachable at recording time: the TSA URL, hash algorithm, and the base64-encoded timestamp token.
- The ECDSA signature over the canonical JSON of all of the above fields, computed once every other field is finalized.
- The signer's public key, in PEM form, so a verifier can check the signature without needing a separate keystore or prior key exchange.

This complete structure is what gets serialized and encrypted with AES-256-GCM (Section 4.3.1) before being written through the four storage mechanisms above. Note that the recording's evidence_id (Section 4.4.6) is deliberately generated afterward, once the package has already been sealed and embedded — it is a locally-assigned lookup key used by the database and reference PNGs, not a field inside the signed metadata itself.

4.5 project_UI.py — Application Logic and Interface

4.5.1 Theme Manager

Applies the user's saved light/dark/system theme preference to the whole application at startup.

4.5.2 Evidence Database

A local SQLite database maintains three tables: a record of every evidence-generating recording (including its unique evidence identifier, video path, frame count, and recorded hash), a verification history log, and the local perceptual-hash registry described in Section 4.4.5.

4.5.3 Recorder

Manages camera device enumeration and selection, drives the actual frame capture and MP4 writing via OpenCV, computes the hash chain and Merkle tree incrementally as frames are captured, and — on stopping — orchestrates building, signing, encrypting, and storing the evidence package through all mechanisms described in Section 4.4.

4.5.4 Location Services

Resolves the recording location using the most precise method available, in order of preference: OS-level location services (the Windows Location API, when the optional winsdk dependency is installed), which are typically accurate to within tens of meters; falling back to approximate IP-based geolocation, accurate only to city/metro level, on other platforms or when precise services are unavailable. Because the precise, GPS-based path returns coordinates but no city/country, a reverse-geocoding lookup (OpenStreetMap Nominatim) fills in a human-readable city and country for display purposes; this lookup is best-effort and never blocks recording if it fails, since only the coordinates themselves are part of the evidence. Per the project specification, location resolution is treated as mandatory: if no location can be determined by either method, recording is refused rather than proceeding without it. An "Open in Maps" link is also available in the interface so a reviewer can visually confirm where a location actually resolved to.

4.5.5 Application Screens

- Record View — camera preview, device selection, and recording controls.
- Verify View — video and evidence selection, passphrase entry, verification results display, PDF report generation, and the evidence export feature described above.
- Reports View — browsing and re-opening previously generated PDF verification reports.
- Settings View — theme, default camera, and other user preferences.

4.5.6 Session Password Convenience ("Remember Me")

The private key that signs every recording's evidence is itself encrypted at rest, unlocked at session start by the user's own password (Section 4.3.1). To spare the user from retyping that password at every launch, an optional "remember my password" feature stores it locally: a Fernet symmetric key is generated once and saved to its own file, and the session password is encrypted with that key and saved to a second file, both restricted to the current user's file permissions where the operating system supports it. On a later launch, the password is silently decrypted and used to unlock the private key without prompting; a "Forget Saved Password" action deletes both files.

This is a deliberate, scoped convenience feature, not a replacement for the session-password-based key encryption itself: it is off by default, decrypting it never requires re-entering the session password, and it is discussed further as a security trade-off in Sections 5.3 and 7.

5. Cryptographic and Security Design

5.1 What This System Proves, and What It Does Not

A successful verification demonstrates that a specific video file's frame content and sequence match exactly what was hashed and signed at the moment recording stopped, on this installation, using a key that installation controls. It is a strong technical guarantee against post-recording alteration of the file.

It does not, by itself, prove that the original camera feed was a genuine, live capture of a real scene rather than a fabricated or pre-recorded source fed into the same capture pipeline — for example, via virtual camera software. This is a structural limitation shared by every software-only capture-authentication system, including the C2PA standard in its current form, and is the primary open direction for future hardening (see Section 8).

5.2 Cryptographic Primitives Used

Purpose	Algorithm / Parameters
Digital signatures	ECDSA on curve SECP384R1
Metadata encryption	AES-256-GCM (random nonce and salt per operation)
Password-based key derivation	PBKDF2-HMAC, 390,000 iterations
Frame and chain hashing	SHA-256
Perceptual (visual-similarity) hashing	64-bit dHash
Erasure coding	Reed–Solomon over GF(256), 6 data + 4 parity chunks (default)
Trusted timestamping	RFC 3161 (via a public Time-Stamp Authority)

Table 5.1 — Cryptographic primitives and their purpose.

5.3 Known Limitations

18. Root of trust: the system cannot distinguish a genuine physical camera from a virtual camera device presenting fabricated footage to the same capture API.

19. Embedded evidence does not survive re-encoding/transcoding, by the nature of how transcoding rewrites a file's bytes from scratch; the companion file and Reed–Solomon chunks exist specifically to mitigate this.
20. Perceptual hashing is a similarity estimate, not a cryptographic guarantee, and can be affected by lighting, color grading, or camera processing; it is used only as a supporting, not a primary, signal.
21. RFC 3161 timestamping requires network connectivity at recording time; if unavailable, the rest of the evidence pipeline still functions normally.

22. The optional “remember my password” convenience feature (Section 4.5.6) stores the session password locally, encrypted with a key that is itself stored unencrypted on disk; anyone with file-system access to that installation can therefore recover the saved password. This is a deliberate, disclosed convenience/security trade-off, off by default, rather than a flaw in the core signing or encryption design.

6. Testing and Evaluation

6.1 Objective

Testing was designed to evaluate the robustness of Project Crygan in preserving forensic evidence integrity and detecting different forms of tampering, specifically covering: secure evidence generation, multi-path evidence storage, evidence recovery mechanisms, cryptographic verification, frame-level integrity checking, tamper detection and localization, and chain-of-custody preservation.

Four representative scenarios were tested, covering both direct video manipulation and manipulation of a secondary evidence artifact.

6.2 Test Case 1 — Original Video Verification

Procedure

A video was recorded using Project Crygan. During recording, the system generated embedded evidence inside the MP4 file, a companion .crygan package, an LSB reference PNG, Reed–Solomon recovery PNGs, a trusted timestamp, and the associated cryptographic signatures. The original, unmodified video was then verified.

Result

Verification Stage	Outcome	Notes
Embedded evidence recovery	**Passed**	Recovered directly from the video file
Metadata decryption	**Passed**	AES-256-GCM decryption successful
Digital signature	**Passed**	ECDSA signature verified
Frame hash chain	**Passed**	Recomputed chain matched recorded value
Merkle commitment	**Passed**	Root matched; no frame-level discrepancies
LSB reference	**Passed**	Independent cross-check confirmed
Overall integrity	**PASSED**	

Table 6.1 — Test Case 1 results.

This confirmed that the evidence generation and verification pipeline functions correctly end-to-end when the recording remains unchanged.

6.3 Test Case 2 — Trimmed Video

Procedure

The recorded video was manually trimmed using a third-party video editor. This re-encoding process removed the embedded evidence from the MP4 (as expected — see Section 5.3), but the companion .crygan file and LSB reference PNG were retained.

Result

Verification Stage	Outcome	Notes
Evidence recovery	**Passed**	Recovered from the companion .crygan package
Metadata decryption	**Passed**	
Digital signature	**Passed**	
LSB reference	**Passed**	
Frame hash chain	**Failed**	Recorded: 236 frames — Trimmed: 160 frames
Merkle commitment	**Failed**	Root mismatch, consistent with frame removal
Overall integrity	**FAILED**	Correctly flagged as altered

Table 6.2 — Test Case 2 results.

Perceptual analysis of the surviving frames was consistent with ordinary re-encoding rather than pixel-level visual manipulation, which is the expected signature of a trim/re-encode operation rather than deliberate visual tampering.

Interpretation

This demonstrates that Project Crygan can recover evidence even when the embedded copy is lost, and correctly identifies frame removal — distinguishing it, via the frame count and perceptual analysis together, from ordinary transcoding of otherwise-unaltered content.

6.4 Test Case 3 — Exposure and Saturation Editing

Procedure

The recorded video was edited to modify its exposure and saturation, without adding or removing any frames. The companion .crygan file and LSB reference PNG were preserved.

Result

Verification Stage	Outcome	Notes
Metadata decryption	**Passed**	
Digital signature	**Passed**	
LSB reference	**Passed**	
Frame hash chain	**Failed**	Both videos: 236 frames — sequence intact

Verification Stage	Outcome	Notes
Merkle commitment	**Failed**	Root mismatch localized to affected frames
Overall integrity	**FAILED**	Correctly flagged as altered

Table 6.3 — Test Case 3 results.

Perceptual analysis classified almost all frames as visually similar to the originals, correctly reflecting that the edit was a global color/exposure adjustment; one frame exceeded the configured similarity threshold, correctly flagging a detectable visual alteration.

Interpretation

This demonstrates that the system detects global visual modifications — such as exposure and color grading — even when the temporal (frame-count and ordering) structure of the video is left completely intact, a case the frame-count check alone could not catch.

6.5 Test Case 4 — LSB Reference PNG Tampering

Procedure

The original video and its companion .crygan file were left completely unchanged. Only the LSB reference PNG was edited (exposure, saturation, and color correction applied), and this modified PNG was supplied during verification.

Result

Verification Stage	Outcome	Notes
Embedded evidence	**Passed**	
Metadata decryption	**Passed**	
Digital signature	**Passed**	
Frame hash chain	**Passed**	
Merkle commitment	**Passed**	
LSB reference check	**Failed**	"No valid Project Crygan LSB reference found in this frame (magic header mismatch)"
Overall integrity	**PASSED**	Video correctly identified as genuine

Table 6.4 — Test Case 4 results.

Interpretation

Editing the reference PNG destroyed its hidden LSB data, causing that specific cross-check to fail — but because the video itself still contained valid, unaltered embedded evidence, all cryptographic verification of the actual recording succeeded, and the video was correctly identified as genuine. Only the independent corroboration normally provided by the reference PNG was lost.

This result validates the intended layered design of the system: corruption or tampering of a secondary, corroborating evidence artifact does not invalidate an authentic primary evidence package, and the system correctly reports the two as separate findings rather than conflating them.

6.6 Evidence Recovery Evaluation

Recovery behavior was evaluated by intentionally removing embedded evidence through video editing. Project Crygan attempts recovery in the following priority order: embedded evidence within the video, the companion .crygan package, Reed–Solomon recovery PNGs, and finally the local out-of-band registry. Testing confirmed that the verifier successfully recovered evidence from the companion package whenever the embedded copy was unavailable, allowing verification to proceed without interruption. Reed–Solomon chunk-frame recovery (Section 4.4.4) and the local registry fallback (Section 4.4.5) were not exercised in this round of testing, since a companion file was always available once the embedded copy was removed. Reed–Solomon recovery is implemented but was not fully evaluated in the current experimental study; an end-to-end test that deletes both the embedded copy and the companion file, reconstructs the evidence package from the Reed–Solomon PNGs alone, and verifies the result is identified as future work (Section 8.2).

6.7 Multi-Layer Verification Summary

Verification Layer	Purpose	Test Outcome
Embedded evidence	Primary evidence source	**Verified**
Companion .crygan file	Recovery when embedded evidence is unavailable	**Verified**
AES metadata decryption	Secure metadata recovery	**Verified**
ECC digital signature	Evidence authenticity	**Verified**
SHA-256 frame hash chain	Detects frame modifications	**Verified**
Merkle tree	Frame-level localization	**Verified**
LSB reference PNG	Independent corroboration	**Detects PNG tampering**
Trusted timestamp	Recording time authenticity	**Verified**

Table 6.5 — Summary of all verification layers exercised across the four test cases.

6.8 Overall Test Outcome

Across the four scenarios tested, Project Crygan:

- Generated cryptographically protected forensic evidence correctly during recording.
- Stored redundant copies of that evidence through multiple independent mechanisms.
- Recovered evidence successfully when the primary embedded copy was lost.
- Detected video trimming and frame removal, correctly distinguishing it from ordinary transcoding via perceptual analysis.
- Detected global visual modifications, such as exposure and saturation adjustment, even with the frame sequence fully intact.
- Detected tampering of the LSB reference PNG independently, without incorrectly invalidating an authentic underlying video.
- Preserved and correctly reported the authenticity of genuine, unaltered recordings in every applicable test.

7. Current Limitations

The testing and design review conducted for this project surfaced the following limitations, which represent honest, practical boundaries of the current implementation rather than defects.

7.1 Embedded evidence may be removed during editing

Most video editing software rewrites the MP4 container during export, causing the embedded evidence package to be removed. The companion .crygan package and Reed–Solomon recovery mitigate this, but the embedded copy itself cannot always survive re-encoding.

7.2 Companion evidence must be preserved

If both the embedded evidence and the companion .crygan package are lost, recovery depends entirely on the availability of the Reed–Solomon PNG recovery images. Proper preservation of all exported evidence artifacts remains essential.

7.3 The LSB reference PNG is not a primary evidence source

It is intended only for independent corroboration. Editing or recompressing it destroys the embedded data and causes that specific check to fail — but, as demonstrated in Test Case 4, this does not invalidate the authenticity of the original video if the primary evidence package remains intact.

7.4 Perceptual hashing provides corroborating, not cryptographic, evidence

Perceptual hashing estimates visual similarity rather than providing a cryptographic guarantee. Lighting changes, color grading, compression, or camera processing can all alter perceptual hash values; it is therefore used as supporting evidence alongside the stronger cryptographic mechanisms, never as a replacement for them.

7.5 Computational overhead

Generating frame hashes, Merkle trees, perceptual hashes, digital signatures, encrypted evidence packages, and trusted timestamps introduces additional processing overhead compared to conventional video recording. This trade-off is considered acceptable for forensic applications, where integrity is prioritized over raw performance.

7.6 Dependence on an external timestamp authority

Trusted timestamp generation relies on an RFC 3161 Time-Stamp Authority. In environments without network connectivity, timestamp acquisition may be delayed or unavailable, although the rest of the evidence generation process continues to function normally.

7.7 Limited attack coverage in current testing

The evaluation conducted to date covers original-video verification, video trimming, exposure/saturation editing, and LSB reference tampering. Additional testing against frame insertion, frame reordering, localized object removal, AI-generated deepfakes, subtitle or watermark insertion, codec migration, and metadata forgery would provide a more comprehensive assessment of the framework, and is identified as future work.

7.8 No hardware root of trust

As discussed in Section 5.3, the system cannot cryptographically distinguish a genuine physical camera sensor from a virtual camera device presenting fabricated footage into the same capture

pipeline. This is an open, structural limitation shared with other software-only capture-authentication systems, and is not resolved by any mechanism currently in the project.

7.9 Remembered session password is a convenience/security trade-off

As noted in Section 4.5.6, the optional “remember my password” feature stores the session password locally so it does not need to be retyped at every launch. The password is encrypted before being saved, but the key used to encrypt it is itself kept, unencrypted, alongside it on the same machine. This means the feature's protection is against casual inspection of the file's contents, not against anyone with file-system access to that installation; such an attacker could recover both files and obtain the password. Users handling sensitive evidence should weigh this trade-off and may prefer to leave the feature off, re-entering their session password at each launch instead.

8. Conclusion and Future Work

8.1 Conclusion

The experiments conducted demonstrate that Project Crygan provides a layered and resilient forensic verification framework. The system not only detects modifications to recorded videos but also distinguishes tampering of the primary evidence from tampering of auxiliary evidence artifacts — a distinction that matters in practice, since the two carry very different implications for a video's authenticity.

Through the combined use of embedded evidence, the companion .crygan package, the LSB reference PNG, Reed–Solomon recovery, cryptographic signatures, Merkle commitments, and perceptual analysis, the framework maintains evidence authenticity while providing robust mechanisms for both recovery and fine-grained integrity verification. The observed limitations relate primarily to practical deployment considerations and expanded attack coverage, and offer clear, concrete directions for future work rather than open design flaws.

8.2 Future Work

- Extend adversarial testing to frame insertion, frame reordering, localized object removal, AI-generated deepfake substitution, subtitle/watermark insertion, codec migration, and metadata forgery.
- Investigate mitigations for the hardware root-of-trust gap, such as detecting known virtual-camera driver signatures or sensor-noise (PRNU) consistency analysis.
- Publish a formal, empirically evaluated threshold study for the perceptual-hash similarity classifier against a range of real-world re-encoding pipelines.
- Explore a formal benchmark comparison against existing provenance frameworks such as C2PA and ProofMode, using a shared test corpus.
- Conduct an end-to-end Reed–Solomon recovery test: delete both the embedded evidence and the companion .crygan file, reconstruct the evidence package from the erasure-coded PNG reference frames alone, and confirm successful verification, so this recovery path is validated experimentally rather than by design analysis only.

9. References

1. Coalition for Content Provenance and Authenticity (C2PA) — Technical Specification.

2. Guardian Project — ProofMode: Camera app for verifiable, signed photo/video capture.

3. International Bar Association — eyeWitness to Atrocities documentation methodology.

4. NIST FIPS 186-5 — Digital Signature Standard (ECDSA).

5. NIST FIPS 197 — Advanced Encryption Standard (AES).

6. NIST FIPS 180-4 — Secure Hash Standard (SHA-256).

7. IETF RFC 3161 — Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP).

8. Reed, I. S., and Solomon, G. — Polynomial Codes over Certain Finite Fields, Journal of the Society for Industrial and Applied Mathematics, 1960.